
FORKSERVE: BRANCH-AWARE PREFILL PRUNING AND DISAGGREGATED KV FOR AGENTIC LLM SERVING

Anonymous Authors¹

ABSTRACT

Agentic LLM serving forks a shared trunk into short residuals and then keeps one committed path. Prefix caches can reuse that trunk only after the tokens exist and a hash hits, so the first fan-out still copies the trunk once per child. Prefill/decode disaggregation separates time-to-first-token from decode latency, yet the connector transfers the pages the prefiller computed, including residuals that the harness later aborts. Branch cost should be incurred only for the residual that is committed, and only at the minimum-cost stage at which rejection is admissible.

ForkServe makes the branch a serving object. A copy-on-write context tree turns `fork` into an alias of the parent’s KV pages and allocates frames only for the residual. Aborting a sibling releases those frames alone, so live occupancy collapses to the committed spine. On that tree, ForkServe decides for each residual whether GPU prefill runs and whether any KV crosses the disaggregated connector. The decision is a cascade of increasing cost: a block-hash probe reuses published pages, a draft score skips a repeated loop before a kernel starts, other low-score residuals are prefilled and decoded for a short probe that continues to the budget only if the generated prefix clears the threshold, and a transfer gate inserts only the miss tail of a residual that is kept. Decode attends that spine alone. Longest-common-prefix commit binds the prompt presented by the harness. We prove that speculative tokens never enter the sampler, so committed outputs are identical to a baseline that prefills only committed tokens. A two-class scheduler admits speculative prefill only from the budget left after committed requests meet their latency margins, and cancels it when the commit conflicts.

On Qwen3-14B GSM8K, ForkServe matches prefix-cache accuracy at $0.72\times$ the median peak KV, cuts modeled fan-out prefill by 74.72% relative to that cache, and raises the concurrency a fixed pool can sustain by 35.53%. On Qwen3-8B MATH-500, skipping every low-score residual drops any-finished accuracy from 20/32 to 13/32 at fan-out 16; a 128-token probe that continues only when the generated prefix clears the threshold restores 20/32 and cuts end-to-end time from 104.1 s to 72.2 s. On Qwen3-8B GSM8K the prefill bar is frozen at $\tau_{\text{pre}}=0.15$ and is not retuned per decoder: at $k=16$ it keeps 16/16 any-finished, cuts computed prefill by 23%, and reduces ESC time from 17.05 s to 14.14 s. A 0.45 bar is faster at that width (10.93 s, still 16/16) but drops two questions at $k=4$ and $k=8$.

1 INTRODUCTION

LLM serving began as a single-turn problem: a prompt arrives, the engine prefills, and tokens are streamed [47, 18, 53]. Production agents—coding assistants [44, 39], computer-use systems [55], and multi-agent workflows [42, 15]—execute programs of LLM calls interleaved with tools [46, 50]. Tree-of-Thoughts and MCTS expand sibling thoughts that share a prefix and diverge in a short residual [45, 26]. The serving unit is a *branching execution*, not a request.

Prior systems raise the atom from a request to a program [25,

17], a session [20, 43], or a workflow DAG [36, 23, 48]. They share an implicit assumption: the next LLM call is a unique continuation of the current context. After a tool call is emitted, the next prompt is a *distribution* over branches the engine can see *before* the observation returns: known suffixes with $p=1$ (templates, wrappers), fan-out siblings that share a trunk of length L and differ in residuals ℓ_i , and observation residuals with $p<1$. Current engines wait until the chosen branch is fully materialized. Automatic prefix caching (APC) [18, 53] discovers sharing only after tokens exist. Prefill–decode disaggregation [54, 31] isolates TTFT from inter-token latency, but the connector transfers every KV page the prefiller computed. Once KV retention has removed trunk recompute [20, 43], the remaining TTFT term is residual-suffix prefill.

¹Anonymous Institution. Correspondence to: Anonymous Authors <anon@example.com>.

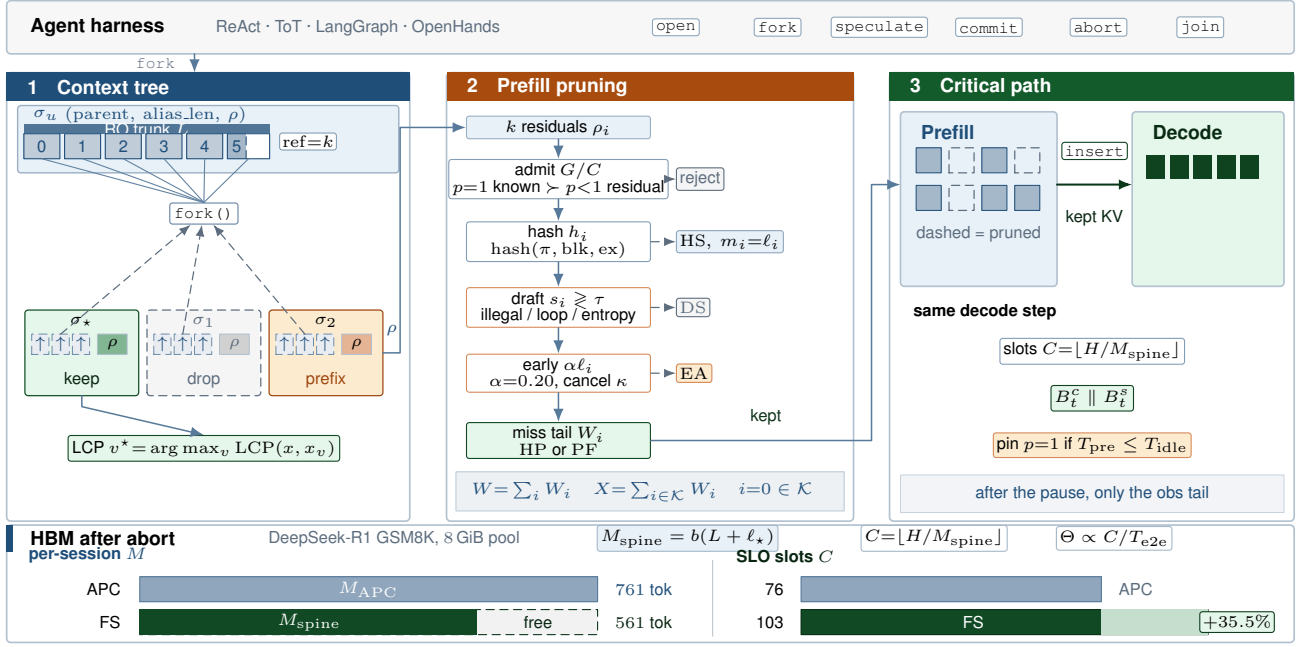


Figure 1. ForkServe architecture. (1) σ_u holds one read-only trunk; `fork` aliases it in $O(1)$; LCP binds the winner. (2) A residual exits at the first decisive action. $m_i = \ell_i$ is HS ($W_i=0$); $0 < m_i < \ell_i$ is HP; a clean low score is DS; the first $\alpha \ell_i$ failing is EA; otherwise PF prefills ℓ_i on the alias. Index 0 stays in $\mathcal{K}$, and X sums W_i only over that kept set. (3) Slots are $C = \lfloor H/M_{\text{spine}} \rfloor$; B_t^c runs beside B_t^s , and a known suffix is pinned when $p=1$ and $T_{\text{pre}} \leq T_{\text{idle}}$. After abort the live occupancy is the spine; the same pool admits 76→103 sessions.

This work. We present ForkServe, a serving system whose first-class object is the *branch* (Figure 1). Three mechanisms are coupled.

(1) A forkable context tree with copy-on-write KV pages (§3.2). Each session is a tree of committed and speculative nodes. `fork` creates a child that inherits the parent’s pages by reference; writes allocate residual blocks. `abort` drops a subtree and decrements residual refcounts only, so live occupancy collapses to the spine $M_{\text{spine}} = b(L + \ell_*)$.

(2) Cost-ordered prefill pruning and transfer gating (§3.4). On top of the tree, each residual ρ_i receives one action $a_i \in \mathcal{A}$ and a pair (W_i, X_i) (Figure 1). A published prefix of length m_i is retrieved: $m_i = \ell_i$ is HS ($W_i = 0$) and $0 < m_i < \ell_i$ is HP ($W_i = \ell_i - m_i$). A miss ($m_i = 0$) has no row in H ; DS and EA reject on $s_i < \tau$ before or inside the first $\alpha \ell_i$ tokens, and PF charges ℓ_i on the trunk aliased at `fork`. Index 0 is forced into $\mathcal{K}$, so the connector inserts only that kept miss tail [54, 31]. LCP commit binds the observed prompt; speculative tokens never enter the sampler.

(3) The spine on the critical path (§3.5). Decode slots are $C = \lfloor H/M_{\text{spine}} \rfloor$. On tick t the budget splits as $B_t = B_t^c + B_t^s$: B_t^c decodes x_* and B_t^s prefills ρ_*^+ in the same step. A known suffix with $p(w)=1$ and $T_{\text{pre}}(|w|) \leq T_{\text{idle}}(u)$ is

pinned during the tool pause; the observation arrives as the unmatched tail.

§4 reports four measurements that are not interchangeable. On Qwen3-14B GSM8K, ForkServe matches APC accuracy at $0.72 \times$ median peak KV; fan-out is still slower than APC. A control-plane study of APP then shows that hash skip, draft prune, and the P/D gate remove most of that prefill on residuals that fail the draft score, and that the measured KV gap raises SLO-feasible concurrency by 35.5%. The GPU rule is narrower than that blanket skip: only a repeated rule is dropped before prefill, and any other low-score residual is probed and extended when the generated prefix clears the threshold (Table 6). On Qwen3-8B GSM8K the prefill bar is a fixed plug-in $\tau_{\text{pre}}=0.15$: it keeps 16/16 any-finished at $k=8$ and $k=16$ while cutting computed prefill by 23% and ESC time from 17.05 s to 14.14 s; a 0.45 bar is faster at $k=16$ but drops two questions at smaller fan-out (Tables 7 and 8). A Qwen3-8B GPU run of ForkServe+ shows the same KV ordering. A six-model forest ($n=40$) keeps GSM8K peak KV at $0.72\text{--}0.75 \times$ APC.

Table 1. Notation for one branching step and for APP.

Symbol	Domain	Meaning
$L, \ell_i, \ell_\star$	$\mathbb{N}$	trunk; residual i ; committed residual
ρ_i, x_u	tokens	residual of child i ; committed trunk
h_i	digest	$\text{hash}(\text{parent}, \rho_i[0:P], \text{extra})$
m_i	tokens	full-page prefix of ρ_i in H , steps of P
s_i, τ	$[0, 1]$	draft score of ρ_i ; prune threshold
α	$(0, 1]$	early-abort prefix fraction (0.20)
p_i, q_i	$[0, 1]$	admit probability; residual-prefix hit rate
$a_i \in \mathcal{A}$	enum	skip / partial / abort / prefill (Eq. 5)
$\mathcal{K}$	$\subseteq [k]$	residuals that remain after APP
W_i, X_i	tokens	GPU miss tail; connector payload
b, P	bytes, tok	KV bytes per token; page size
$B_t = B_t^c + B_t^s$	tokens	two-class batch budget on tick t

2 BACKGROUND AND MODEL

2.1 Prefill, Decode, and Idle

Autoregressive inference has two mechanically different phases [47, 54, 31]. *Prefill* consumes the prompt in parallel and is compute-bound; *decode* emits one token (or a small chunk) per step and is memory-bandwidth bound. PagedAttention [18] stores KV in non-contiguous pages; RadixAttention [53] indexes pages by token prefix so concurrent requests that share a system prompt hit a common trunk. Two consequences matter for agents. First, next-turn TTFT is the prefill of tokens that miss the cache, plus queuing. Second, a memory-bound decode batch rarely saturates the tensor cores that a concurrent chunked prefill could use [2]. Agent serving adds a third fact: the prompt of turn $i+1$ is a *delta* on turn i , applied after a tool pause of highly variable length [20, 43].

2.2 A Branching Step

We treat one agentic step as a rooted prefix tree, not as k independent completions.

Definition 1 (Branching step). A parent node u holds a committed trunk x_u of length L . A harness-declared fan-out produces k children with residuals ρ_i of length ℓ_i , $i \in \{1, \dots, k\}$. After a winner $\star$ is bound, only the spine $x_\star = x_u \parallel \rho_\star$ of length $L + \ell_\star$ remains live.

Table 1 is the symbol set used in the rest of the paper. Let b be bytes of KV per token (all layers, one sequence) and P the page size in tokens. Three occupancies follow immediately:

$$\begin{aligned} M_{\text{clone}} &= b(kL + \sum_i \ell_i), \\ M_{\text{CoW}} &= b(L + \sum_i \ell_i), \\ M_{\text{spine}} &= b(L + \ell_\star). \end{aligned} \quad (1)$$

The first-fan-out work of a scheme is the number of tokens sent to a GPU prefill kernel before the winner is known; the transfer span is the number of tokens a P/D connector ships.

Why APC misses the first fan-out. APC indexes a *full* block by $h = \text{hash}(\text{parent}, \text{tokens}, \text{extra})$ *after* those tokens exist [18]. A same-batch ToT expansion looks up k children before any child has published a hash, so the walk misses and the engine clones k trunks: $W_{\text{APC}} = kL + \sum_i \ell_i$. A later replay of the committed winner is a full hash hit ($W=0$). Hash-only prefill therefore helps the second session, not the first expand.

Why prefill–decode disaggregation does not raise tokens/s. DistServe and Splitwise isolate TTFT from inter-token latency by placing prefill and decode on different instances and moving KV through a connector (`insert` is non-blocking, `drop_select` blocks) [54, 31]. That baseline transfers every page the prefiler computed,

$$X_{\text{PD}} = kL + \sum_i \ell_i, \quad (2)$$

so first-fan-out transfer equals first-fan-out work. P/D is a placement, not a branch filter.

Four operators that generate the tree. Tool intercept [46, 34] idles the engine while the environment runs; a known suffix is available immediately and an observation residual arrives later. Fan-out/join [42, 15] launches k children that share x_u and differ in short role residuals. Search [45, 26] expands b siblings, scores them, and recurses on a shared spine. Context transformation [30] forks a summarized child while the original spine stays abortable. In all four cases the next call is drawn from a discrete set $\mathcal{B}(u)$; existing engines observe only the child that is eventually issued.

Insight 1 (Prune before prefill). APC reuses a trunk only after a hash hit; prefill–decode disaggregation transfers the span that was computed. The missing operation is a per-residual decision (W_i, X_i, a_i), issued *before* a GPU kernel and *before* `insert`.

Insight 2 (Fork by reference). On fan-out the trunk is typically several times the per-child residual. Materializing k independent KV copies allocates $\Theta(k)$ redundant HBM for tensors that attention reads as identical. A location-addressed alias at `fork` makes a miss residual-only.

3 DESIGN

ForkServe is placed between the agent runtime and the LLM engine (Figure 1). The harness announces structure; the control plane admits, prunes, binds, and retains; the backend moves pages and tokens.

3.1 API

<code>open(session)</code>	Create a root; prefill system+user.
<code>fork(u, bid, tok?)</code>	Child of u ; optional known suffix.
<code>speculate(v, pri)</code>	Mark v eligible for speculative prefill.
<code>commit(u, tokens)</code>	Bind by LCP; start decode.
<code>join(children, pol)</code>	Merge fan-out (all/first/kofn).
<code>abort(v) / close</code>	Drop a subtree; end the session.

`commit` is the only verb on the critical path of user-visible tokens.

3.2 Forkable Branch State

Definition 2 (Context tree). A session is a rooted tree $T = (V, E)$. Node u stores a token sequence x_u such that $x_{\pi(u)} \preceq x_u$, a residual $\delta_u = x_u[|x_{\pi(u)}|:]$, a mode $\rho_u \in \{\text{SPEC}, \text{COMMIT}, \text{IDLE}, \text{DEAD}\}$, a generation g_u , and a logical page table $\sigma_u = (\text{parent}, \text{alias_len}, \rho_u^{\text{pages}})$.

Invariants: (i) prefix; (ii) at most one committed child per node except during join; (iii) speculative nodes are invisible to sampling; (iv) a page’s refcount equals the number of non-dead nodes that map it. `fork(u, bid, xk)` copies two words ($\text{parent} \leftarrow \sigma_u$, $\text{alias_len} \leftarrow |x_u|$) and increfs the aliased frames— $O(1)$ in L . A write that hits $\text{ref} > 1$ or a read-only tail CoW-copies only the valid rows. Figure 2 is that pool. A multi-level fan-out is a *sequence* of CoWs (Figure 4): each edge aliases the parent and allocates only $+r$; live paths share every page on the common prefix; abort is residual-only, and a subtree with no live child cascades until the committed spine. The two compose: LCP commit publishes full winner pages into the APC index so a later session can hash-hit them (§3.4). Abort frees $b\ell_i$ bytes, not $b(L + \ell_i)$. TTL and offload are node-granular: speculative leaves first, then idle committed leaves, then the spine [20, 43].

3.3 Admission

Candidates $\mathcal{B}(u)$ come from harness-declared wraps, grammar mass over the next tool name (top- m), and a session-local prior of ($\text{parent_role}, \text{bid}$). Priors never invent a branch id. Each candidate is a pair $(x_b^k, \hat{x}_b^o)$: an exact known suffix and an optional observation guess. This is *input-side* speculation—a cache of a future prompt, not draft output tokens [19, 7].

The expected TTFT reduction and the interference-aware cost of a work item w are

$$G(w) = p(w) \cdot \min(T_{\text{pre}}(|w|), T_{\text{idle}}(u)) \cdot \eta(w), \quad (3)$$

$$C(w) = b \cdot |w| + \lambda \cdot \max(0, T_{\text{pre}}(|w|) - \gamma), \quad (4)$$

where η down-weights residuals by q_b and work that would finish after the pause, and λ matches 1 ms TBT to 4 ms TTFT [52].

Proposition 1 (Known-suffix first). *If two items have equal T_{pre} and w is a known suffix ($p=1, q=1$) while w' is an observation residual ($q < 1$), then $G(w)/C(w) \geq G(w')/C(w')$.*

Admission is a greedy fractional knapsack on G/C subject to idle horizon, residual HBM, branch cap m , and committed TBT margin. Chunks of size $\leq C_{\text{spec}}$ make a mid-prefill `commit` discard at most one in-flight chunk.

LCP commit. When the harness presents prompt x of parent u : (1) $v^* = \arg \max_v \text{LCP}(x, x_v)$ among live children; (2) pages covering the LCP become committed (CoW-split if mid-page); (3) the unmatched tail is a committed prefill; (4) sibling Spec nodes abort; (5) $g_u \leftarrow g_u + 1$ and decode is released. Speculative jobs tagged $g \neq g_u$ drop in $O(1)$.

Theorem 2 (Output identity). *For any session, the committed decode tokens of ForkServe equal those of a baseline that prefills each committed prompt from scratch (or from a prefix cache of committed tokens only), given equal seeds and weights.*

Proof sketch. Decode at a committed node u attends only to KV(x_u). That KV is produced by a committed prefill of x_u , or by a speculative prefix verified token-for-token by LCP plus a committed tail. Transformer KV of a prefix depends only on that prefix. Speculative nodes do not enter the sampler. $\square$

3.4 Cost-Ordered Prefill Pruning and Transfer Gating

Copy-on-write fixes trunk placement. It does not decide which residual enters a GPU kernel, nor which KV crosses the prefill–decode connector. APP makes both decisions per residual, (W_i, X_i) , by a cascade whose stage cost is strictly increasing (Figure 3). The stages are a block-hash probe, a CPU draft score, a prefix of the prefill kernel, the miss-tail prefill, and a connector `insert`. Stage $i+1$ runs only when stage i does not reject ρ_i . The committed winner is exempt: index 0 is forced into $\mathcal{K}$, speculative pages stay unpublished until LCP commit, and Theorem 2 is unchanged. CoW remains the storage substrate; APP is the filter on compute and on transfer. On the Qwen3-14B GSM8K forest, copy-on-write already stores less KV than APC, yet median fan-out is 140 ms against APC’s 87 ms: the k residuals are still submitted to prefill. The cascade is the stage that removes that work.

The cascade as an action. The order above is one action per residual. Write $\mathcal{A} = \{\text{HS}, \text{DS}, \text{EA}, \text{HP}, \text{PF}\}$ for hash skip, draft skip, early abort, hash partial, and prefill. Let m_i be the longest prefix of ρ_i covered by full pages of the published index H , so $m_i \in \{0, P, 2P, \dots\}$ and $m_i \leq \ell_i$. The probe key is $\text{trunk} \parallel \rho_i$, but the trunk itself is the alias

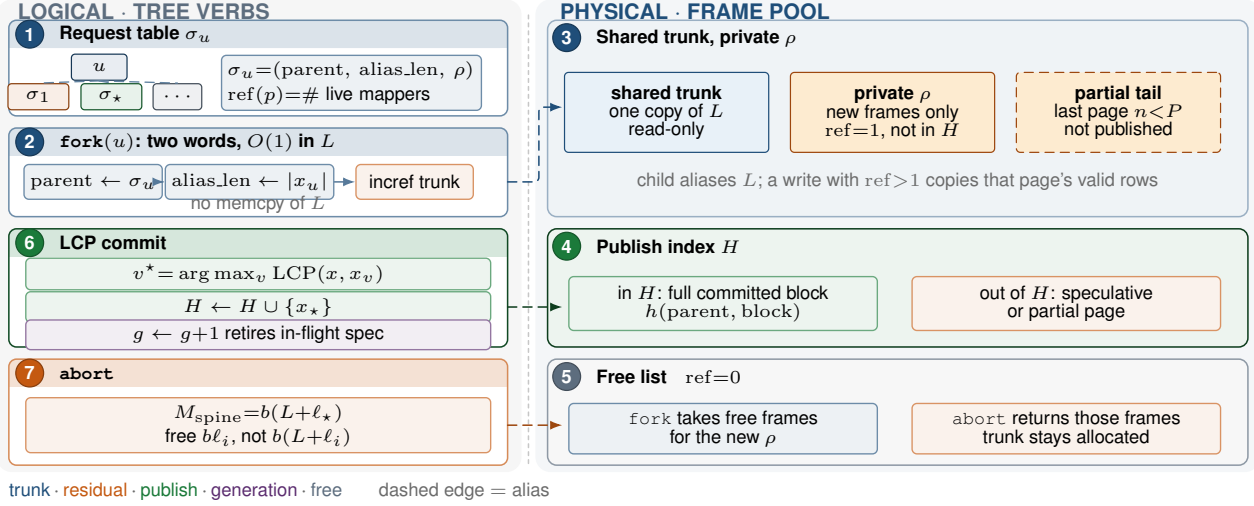


Figure 2. **Context-tree page pool.** `fork` aliases the trunk in $O(1)$ and allocates only ρ ①–③. LCP commit publishes x_* into H ④⑥. `abort` returns residual frames only, so live occupancy is the spine $M_{\text{spine}}=b(L+\ell_*)$ ⑤⑦.

written by `fork` ($\sigma_i \leftarrow (\sigma_u, |x_u|)$), so L is absent from W_i and $W_i \leq \ell_i$ on every action. Cases below are first-match.

$$a_i = \begin{cases} \text{HS} & m_i = \ell_i, \\ \text{HP} & 0 < m_i < \ell_i, \\ \text{DS} & m_i = 0 \text{ and } \rho_i \text{ is a repeated loop,} \\ \text{EA} & m_i = 0 \text{ and the first } \alpha \ell_i \text{ already fails,} \\ \text{PF} & \text{otherwise } (m_i = 0). \end{cases} \quad (5)$$

Winner $i=0$ is forced to $\mathcal{K}$. GPU work and connector payload are

$$W_i = \begin{cases} 0 & a_i \in \{\text{HS}, \text{DS}\}, \\ \lceil \alpha \ell_i \rceil & a_i = \text{EA}, \\ \ell_i - m_i & a_i = \text{HP}, \\ \ell_i & a_i = \text{PF}, \end{cases} \quad (6)$$

$$X_i = \begin{cases} W_i & i \in \mathcal{K} \wedge a_i \in \{\text{HP}, \text{PF}\}, \\ 0 & \text{otherwise.} \end{cases} \quad (7)$$

A full hit is a retrieval: $m_i = \ell_i$ reads published KV and issues no kernel ($W_i = X_i = 0$). A proper prefix retrieves m_i tokens and prefills the uncovered tail $\ell_i - m_i$. A miss ($m_i = 0$) has an empty probe. The draft score runs, and a residual that remains is a prefill of ℓ_i on the aliased trunk. Figure 5 is that rule on the residual “find the dragon.” The dragon head is the radix root on every row. A green prefix is a published page and is spliced onto decode; an orange tail is grown by prefill. Speculative pages stay out of H until LCP commit, so the first expand of an unpublished child has $m_i = 0$ and is the PF row.

Hash-local hits stay on the decode instance (the same published blocks are APC-visible there). Draft-skipped and early-aborted residuals never enter the buffer. Algorithm 1 is the per-fan-out plan; Table 2 is the first-fan-out span.

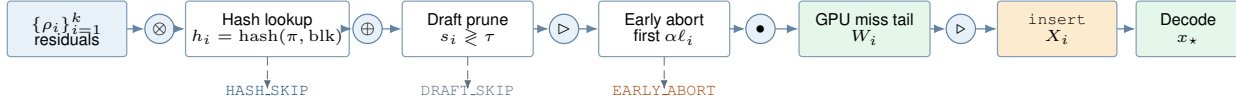
Lemma 3 (First-fan-out dominance). *On a cold index, $W_{\text{APP}} \leq W_{\text{CoW}} \leq W_{\text{APC}} = W_{\text{PD}}$ and $X_{\text{APP}} \leq X_{\text{PD}}$, with strict inequality in W whenever at least one repeated loop is draft-skipped, and strict inequality in X whenever $\mathcal{K} \neq [k]$ or any kept residual is a hash hit.*

Block-hash pruning. The first stage is a table probe and issues no model FLOPs. The index key is `hash(parent, block_tokens, extra)`, with `extra` carrying LoRA id, multimodal hash, or `cache_salt` on block 0 [18]. Only full pages enter H , which is why (5) tests m_i in steps of P . LCP commit inserts full blocks of x_* , which is what makes a later session a hash hit ($m_i = \ell_i$) rather than another fan-out. `fork` itself does not walk hashes: aliasing stays $O(1)$, and the lookup is a filter on the residual, not part of creating the child. A hash-local hit is also not a transfer. The same published blocks are already APC-visible on the decode instance, so $X_i=0$ even though the residual is kept.

Draft-score rejection. On a miss ($m_i = 0$ in (5)) the next stage is a CPU score, still before the prefill kernel. Each residual receives a score from illegal-token and repetition checks, token entropy, or an optional draft model. An earlier draft of this rule treated every score below τ as a draft skip. That rule is not admissible. A residual the text score marks illegal still solves contest items once it is decoded, while a repeated loop solves none (Table 6). Draft skip is therefore

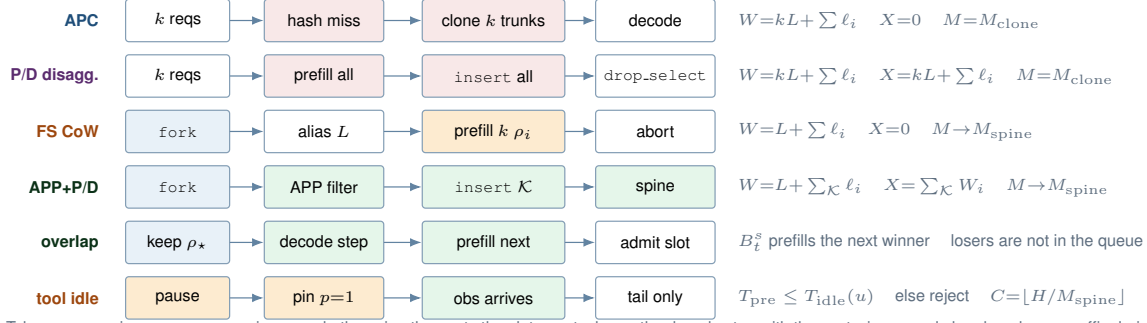
ρ_i : i -th residual h_i : block hash s_i : draft score α : early-abort fraction κ : chunk W_i : GPU miss tail X_i : connector tokens a_i : action

(a) APP + P/D pipeline (Alg. 1)



Parallelism: k residuals Layout: location-addressed CoW, not content-addressed APC Winner $i=0$ is never dropped Spec pages unpublished
Work $W = \sum_i W_i$ Transfer $X = \sum_{i \in \mathcal{K}} W_i$ Occupancy after abort $M_{\text{spine}} = b(L + \ell_*)$ Replay of x_* : $W = X = 0$

(b) First-fan-out dataflow (same k, L, ℓ). Bottleneck in red.



Takeaway: pruning removes non-winner work; the spine then sets the slot count, shares the decode step with the next winner, and absorbs a known suffix during the tool pause.

Figure 3. Cost-ordered prefill pruning and the critical path. (a) A residual exits at the first decisive stage. Only a kept miss tail reaches insert. (b) APC and baseline disaggregation clone the trunk. CoW aliases L . APP keeps the winner. The overlap row prefills that next winner in the decode step. The tool-idle row pins a known suffix during the pause and leaves the observation tail.

Table 2. Work, transfer, and memory spans (bottleneck in red). W_{pre} is GPU prefill tokens; X is connector tokens; M is live KV after abort; T_{ab} is abort cost in pages; replay is a later open of the published winner. PAR is the number of trunk copies touched on the first expand.

	W_{pre}	X	M after abort	T_{ab}	PAR _{trunk}	Replay W_{pre}
Recompute	$kL + \sum_i \ell_i$	0	$kL + \sum_i \ell_i$	$\Theta(kL/P)$	k	$kL + \ell_*$
APC (first expand)	$kL + \sum_i \ell_i$	0	$kL + \sum_i \ell_i$	LRU of hashed ρ	k	0
hash prefill (first)	$kL + \sum_i \ell_i$	0	$kL + \sum_i \ell_i$	LRU	k	0
P/D disagg.	$kL + \sum_i \ell_i$	$kL + \sum_i \ell_i$	$kL + \sum_i \ell_i$	full KV transfer	k	0
FS CoW	$L + \sum_i \ell_i$	0	$L + \ell_*$	$\Theta(\ell_i/P)$	1	L_{miss}
FS+ + P/D	$L + \sum_{i \in \mathcal{K}} \ell_i$	$\sum_{i \in \mathcal{K}} W_i$	$L + \ell_*$	lazy, residual only	1	0

$P=16$. Known-suffix items ($p=1$) are admitted before residuals ($p < 1$). Spec pages are never hashed; LCP commit publishes full blocks of x_* .

only the repeated loop: that residual is not submitted to prefill and is not inserted as a hash key. The prefill bar is a fixed plug-in, $\tau_{\text{pre}}=0.15$, and is not retuned when the host decoder is ESC, Speculative Rejection, or DPTS. The winner, and every other residual whose score is at least τ_{pre} , is prefilled and decoded to the budget D . A non-winner that scores below τ and is not a loop is still prefilled. It is then decoded for a probe of P tokens, and the same score is applied to the generated prefix $y_{1:P}$. If that score is at least τ , decode continues from P to D ; otherwise it stops at P . The probe is a prefix of decode, not the residual early-abort in panel (f) of Figure 5. On the measurements below, $\tau=0.45$ and $P \in \{64, 128\}$. The control-plane rows in §4.2 still charge the earlier blanket skip, which drops both low-score residuals before prefill.

Gated connector insert. Prefill–decode disaggregation places prefill and decode on separate instances [54, 31]. It does not choose a branch. The connector stays vLLM-shaped—insert is non-blocking, drop_select blocks—and a production pipe (Nixl, LMCake, Mooncake) keeps the same gate. Baseline disaggregation uses (2) and therefore transfers the computed span, including the k -way trunk. APP uses (7): X_i is nonzero only for a kept residual whose action is HP or PF, and then only for the miss tail that the kernel computed. Rejected residuals and hash-local hits stay on the prefill instance, so gated transfer reduces both W and X relative to baseline disaggregation. Let μ be microseconds per transferred token. Fan-out time is

$$T_{\text{fan}} = T_{\text{pre}}(W) + \mu X + T_{\text{CoW}} + T_{\text{abort}}, \quad (8)$$



Figure 4. Staged CoW along many paths, threshold abort, cascade abort, shared prefixes. (a) Each edge is one CoW: the child aliases the parent’s pages and allocates only a residual (+r). Live paths A1b1x, A3a2x, and B2a1y share the trunk; the first two also share +r_A. Fork C has depth 2: at CoW-3 all four leaves die, so abort cascades through C1 and C2, then C; it stops at Root, which still has live children A and B. (b) KV stacks of the three live leaves, layers aligned: a shared layer is one physical page set with refcount > 1, not N copies. Aborting A1b1y or B2a1x frees only that leaf residual; cascade-aborting C also frees +r_{C1}, +r_{C2}, and +r_C.

with T_{CoW} a pointer swap (no KV memcopy) and T_{abort} deferrable when abort is lazy. Under the control-plane model of §4.2 ($\ell \times 12 \mu\text{s}$, $\mu = 2 \mu\text{s}$), (8) is 139 ms for APC, 162 ms for baseline disaggregation, 47 ms for CoW, and 31.4 ms for APP.

3.5 The Spine on the Critical Path

After APP, fan-out is the prefill of the kept residual. End-to-end time is still $T_{\text{fan}} + T_{\text{dec}}$, and T_{dec} is the long term. Three placements put the spine on that sum. A fixed chunk width and a fan-out barrier leave the freed KV and the pruned queue off the path.

Slots follow the spine. Let H be the KV pool after weights. A live tree occupies $M_{\text{spine}} = b(L + \ell_*)$. APC still holds the aborted residuals until they expire, so its live set is the larger M_{APC} . The number of concurrent decode slots is

$$C = \lfloor H/M \rfloor, \quad (9)$$

with $M = M_{\text{spine}}$ for ForkServe and $M = M_{\text{APC}}$ for the prefix cache. C is not a constant chunk size. A request that finishes its current decode step releases the slot, and the next queued spine takes it. Throughput is

$$\Theta \propto C/T_{\text{e2e}}. \quad (10)$$

The same H therefore admits more sessions once occupancy is the spine, and the wall clock of a batch is the number of waves n/C , not n divided by a fixed width.

Prefill the next winner during decode. Each tick t has a token budget B_t from the chunked-prefill cap [2]. JIT-

Serve’s margin [52] gives each committed request r a minimum token rate $\underline{s}_r$.

$$B_t^c = \min\left(B_t, \sum_{r \in R_t^c} \max(\underline{s}_r \Delta, 1)\right), \quad (11)$$

$$B_t^s = B_t - B_t^c. \quad (12)$$

B_t^c is committed decode. B_t^s is not spent on the $k-1$ residuals APP already rejected. It prefills the miss tail of the next winner, in the same step as the running decode. If $B_t^s = 0$, no speculative chunk is admitted. The fan-out of losers is not a generate that must finish before any decode token. A tree-sticky router pins the trunk to the worker that prefills the root; only residual work may migrate.

Hide the known suffix in the tool pause. A candidate with $p=1$ is an exact wrapper, not a guess. If $T_{\text{pre}}(|w|) \leq T_{\text{idle}}(u)$, that prefill runs while the tool is in flight, including when the parent is still marked committed. The pause is the deadline. A suffix that would finish after $T_{\text{idle}}(u)$ is rejected, as is any observation residual the pause cannot cover. When the observation arrives, LCP commit checks the pinned prefix and the committed prefill is the unmatched tail. Under P/D the same suffix is prefiller work tagged soft-deadline = $T_{\text{idle}}(u)$ and is inserted once, onto the trunk owner. Speculative tokens still do not enter the sampler, so Theorem 2 is unchanged.

4 EVALUATION

The GPU measurements share one node and one engine, and they differ by model, tensor-parallel width, and workload.



Figure 5. Seven cuts of “find the dragon.” The dragon head is the radix root; the body is one residual. Green tokens are full pages in H and are spliced onto decode. Orange tokens are absent from H and are grown by prefill. Gray tokens are withheld. Chips stand in for tokens: a real hit ends on a full page, and each row is a different value of m_i . (a) The body is a hit (HS , $m_i = \ell_i$, $W_i = 0$). (b) The front half “dra” is in the tree and “gon” grows (HP , $W_i = \ell_i - m_i$). (c) The cut falls after “d”. (d) The cut falls before the last character. (e) A miss grows the body from the neck (PF , $m_i = 0$, $W_i = \ell_i$); the head stays the alias. (f) The first $\lceil \alpha \ell_i \rceil$ tokens grow and the kernel stops (EA). (g) A repeated loop is withheld (DS , $W_i = 0$). A low score that is not a loop is not this row: it is prefilled, then decoded for a probe (Algorithm 1).

§4.1 is the full GSM8K forest on Qwen3-14B at $tp=2$. §4.2 is a control-plane cost model of APP, with no GPU, against APC, hash prefill, baseline prefill–decode disaggregation, ForkServe, and three decoding-time pruners. §4.2 freezes the GSM8K score bar at $\tau_{pre}=0.15$ on Qwen3-8B and reports the accuracy–latency trade-off as a plug-in on ESC, Speculative Rejection, and DPTS. §4.3 runs ForkServe+ on Qwen3-8B at $tp=2$ and $tp=4$. §4.4 is a three-turn Tree-of-Thoughts on Qwen3-4B and Qwen3-8B, one problem per generate. §4.5 repeats the forest on six families at $n=40$. §4.6 sweeps branching, depth, pruning, and batch width on Llama-3-8B, Mistral-7B, and DeepSeek-R1-Distill-Llama-8B, then runs a 2048-token contest forest on that DeepSeek model and on Qwen3-4B.

Setup. GPU runs use vLLM on four A100-SXM4 40GB GPUs, with `bfloat16` weights and $tp=2$ or 4. The models are Qwen3-14B, Qwen3-8B, Qwen3-4B, Qwen2.5-Math-1.5B, Llama-3-8B, Mistral-7B, and DeepSeek-R1-Distill-

Llama-8B. Tree-of-Thoughts uses $k=4$ and a 512-token budget (2048 on the contest forest), in batches of eight. The three-turn run in §4.4 issues one problem per generate. A math answer is correct when its final number matches the reference.

4.1 GPU forest

Setup. This section uses Qwen3-14B at $tp=2$. The GSM8K test set has 1319 problems. Each problem is expanded with four thoughts and decoded for at most 512 tokens, in 165 batches of eight. Accuracy is the fraction of problems whose final number matches the reference.

Storage and expand latency. Table 3 reports the storage result and the expand-latency gap that APP is designed to close. Median peak KV is $0.72\times$ APC and $0.25\times$ recompute, and accuracy stays within one point of both. Storage and fan-out do not share that ordering: median fan-out is

Algorithm 1 PrefillPlan($\{\rho_i\}, H, \tau, P, \text{winner}$)**Input:** residuals, hash index H , threshold τ , probe length P **Output:** actions a_i , work W_i , transfer X_i

```

1: for  $i = 1$  to  $k$  do
2:    $m_i \leftarrow H.\text{lookup}(\text{trunk}[\rho_i])$  {full pages of  $\rho_i$ }
3:   if  $m_i = \ell_i$  then
4:      $a_i \leftarrow \text{HS}; W_i \leftarrow 0$  {retrieve}
5:   else if  $0 < m_i < \ell_i$  then
6:      $a_i \leftarrow \text{HP}; W_i \leftarrow \ell_i - m_i$ 
7:   else
8:      $s_i \leftarrow \text{DraftScore}(\rho_i)$  { $m_i = 0$ , no row to read}
9:     if  $i = \text{winner}$  or  $s_i \geq \tau$  then
10:       $a_i \leftarrow \text{PF}; W_i \leftarrow \ell_i$  {decode to budget  $D$ }
11:     else if IsLoop( $\rho_i$ ) then
12:       $a_i \leftarrow \text{DS}; W_i \leftarrow 0$  {repeated loop}
13:     else
14:       $a_i \leftarrow \text{PF}; W_i \leftarrow \ell_i$  {probe  $P$  tokens}
15:      if DraftScore( $y_{1:P}$ )  $\geq \tau$  then
16:        continue decode to  $D$ 
17:      else
18:        stop at  $P$ 
19:      end if
20:    end if
21:  end if
22:  if  $i = \text{winner}$  or  $a_i \in \{\text{HS}, \text{HP}, \text{PF}\}$  then
23:    keep  $i$ 
24:  end if
25:  gate insert by (7)
26: end for

```

Table 3. GSM8K test, Qwen3-14B, tp=2, $n=1319$. Median over 165 batches of eight. Best storage in green.

	recompute	APC	FS
Accuracy	0.847	0.844	0.845
Peak KV, median (tok)	4136	1430	1030
Peak KV, max (tok)	4944	1632	1232
Peak / APC	$2.89\times$	$1.00\times$	$0.72\times$
Fan-out, median (ms)	326	87	140
Generation (min)	—	18.6	18.9
KV saving vs. rec.	—	0.65	0.75

140 ms for ForkServe and 87 ms for APC, because copy-on-write and abort execute on the expand path, whereas an APC hash hit does not. End-to-end generation remains within 2% of APC (18.9 vs. 18.6 minutes). The same peak ordering, $0.58\text{--}0.80\times$ APC, holds on seven math sets for Qwen3-14B, Qwen3-8B, and DeepSeek-R1-Distill-Llama-8B at tp=2 and tp=4. The peak is unchanged when the tensor width doubles; it follows the committed path, as (1) requires.

4.2 Prefill pruning

Setup. Control plane, no GPU. Ten sessions, $k=4$, $L=256$, $\ell=32$. Two of four residuals fail the draft score; the winner is index 0. Prefill time is $\ell \times 12 \mu\text{s}/\text{token}$; P/D transfer is $2 \mu\text{s}/\text{token}$. APC and hash prefill publish blocks only after a request materializes them, so the first fan-out clones k trunks. Baseline disaggregation uses that prefill and then transfers the live token span. ForkServe aliases

Table 4. Control-plane fan-out (10 sessions, $k=4$, $L=256$, $\ell=32$). Bottleneck in red; ForkServe+ in green. Not GPU wall-clock.

Phase	Method	W	T_{fan}	X	peak	skip	prune
fan-out	APC	11520	139.0	0	11520	0	0
	hash	11520	139.0	0	11520	0	0
	P/D disagg.	11520	162.1	11520	11520	0	0
	FS	3840	47.1	0	3840	0	0
	ForkServe+	2592	31.4	32	2880	9	30
ForkServe+ / APC		−77.5%	−77.4%	—	−75%	—	—

Table 5. Decoding-time pruning on the Table 4 fan-out (10×4 , $L=256$). Decode-to-cut is the prefix that must be generated before a child can be dropped. “On losers” counts that prefix on the two low-score children. Same $12 \mu\text{s}/\text{token}$ clock. Not GPU wall-clock.

	prefill	decode	on losers	peak	T_{cut}
ESC	2560	20480	10240	23040	276.5
Spec. Rejection	2560	10240	5120	12800	153.6
DPTS	2560	4000	2000	6560	78.7
ForkServe+	2592	0	0	2880	31.4

the trunk by pointer swap and prefills the k residuals. APP adds hash skip, draft prune, early abort, lazy abort, and a connector that inserts only retained residuals.

Fan-out. Table 4 is the comparison the GPU gap motivates. On the first expand, hash prefill matches APC: the blocks are unpublished, so the hash probe does not remove the cloned trunks. Baseline disaggregation exceeds APC (162 vs. 139 ms) because it transfers that clone. ForkServe removes the clone by aliasing the trunk (3840 prefill tokens, 47 ms). APP then withholds prefill from 30 of the 40 residuals, which reduces T_{fan} to 31.4 ms and the connector payload from 11520 tokens to 32.

Decoding-time pruning. Table 5 runs ESC [21], Speculative Rejection [35], and DPTS [12] on this same fan-out. Each method prefills the shared trunk once. The length of the decode that produces the prune score is the length published for that method. ESC stops after a window of 5 agreeing samples; this fan-out has four children and two low-score answers, so the window stays open and every child is decoded to the 512-token budget. Speculative Rejection is charged its partial-reward horizon $\tau=256$ and then drops the lower half ($\alpha=0.5$); the memory-limit trigger in that paper would not fire at $k=4$, so τ is the earlier checkpoint. DPTS early-stops after its 100-token mini-step. The two low-score children are the ones removed, and the winner is kept. Decoded tokens use the same $12 \mu\text{s}/\text{token}$ clock. DPTS is the earliest cut of the three and still decodes 4000 tokens before the drop, 2000 of them on losers, reaching the cut at 78.7 ms. APP has already withheld those residuals: 0 tokens on losers and $T_{\text{fan}}=31.4$ ms. That row is the blanket skip. §4.2 replaces it: only a repeated loop is withheld before prefill.

Table 6. MATH-500 level ≥ 4 , Qwen3-8B, $n=32$, $D=1024$. Any-finished is exact match on a branch that ran the full budget. “Drop $s < \tau$ ” withholds every low-score residual before prefill. “Grow P ” skips loops, prefills the other low-score residuals, and continues a probe of P tokens to D only if the generated prefix scores at least $\tau=0.45$. Time is trunk warm plus fan-out, in seconds.

	$k=4$	$k=8$	$k=16$
Full fan-out	12/32, 25.3	15/32, 52.5	20/32, 104.1
Drop $s < \tau$	6/32, 19.1	11/32, 25.6	13/32, 52.1
Skip loops only	11/32, 23.0	17/32, 41.5	16/32, 71.4
Grow $P=64$	8/32, 24.0	16/32, 42.0	18/32, 72.5
Grow $P=128$	9/32, 24.7	13/32, 43.3	20/32, 72.2

Probe-then-grow. The blanket skip is the wrong cut once the residual is not a loop. We measure the rule in Algorithm 1 on Qwen3-8B, one GPU each, four A100-40GB, $\text{tp}=1$. The slice is the first 32 MATH-500 problems at level 4 or 5. Temperature is 0, EOS is ignored, prefix caching is on, and the decode budget is $D=1024$. Half of the k residuals are contest strategies. The other half alternates a repeated loop with illegal text. A question counts when some branch that ran all 1024 tokens matches the gold answer. The winner is residual 0.

Table 6 is the correction. Dropping every residual below τ cuts any-finished from 12/32 to 6/32 at $k=4$ and from 20/32 to 13/32 at $k=16$. Loop branches are 0 for 0 at every k , so skipping them before prefill is safe. At $k=16$, a 128-token probe extends every illegal residual (128/128) and none of the loops. Any-finished returns to the full fan-out, 20/32, while end-to-end time falls from 104.1 s to 72.2 s and decode tokens from 524288 to 393216. Probing the loops as well extends none of them and costs 74.4 s for the same 20/32. At $k=8$ the shorter probe is the one that holds accuracy: grow at $P=64$ is 16/32 in 42.0 s, against 15/32 in 52.5 s for the full fan-out. At $k=4$ the probe does not recover the full fan-out (9/32 against 12/32); skipping the single loop, and decoding the illegal residual to D , is 11/32 in 23.0 s.

Fixed score bar, accuracy–speed trade-off. The MATH-500 probe still uses a decode bar $\tau=0.45$ on the generated prefix. Prefill admission is a different score and a different, frozen bar $\tau_{\text{pre}}=0.15$: it drops only the repeated loop (0.02) and starts illegal text (0.22) and every live strategy (1.0). We measure that plug-in on Qwen3-8B GSM8K, $n=16$, $D=512$, the same hopeless mix, four A100-40GB, $\text{tp}=1$. The host decoder is ESC, so a withheld child never pays residual prefill or a decode slot. A denser sweep of the bar collapses to three admission sets; 0.15 and 0.45 are the two nonzero operating points.

Table 7 is the trade-off. The frozen plug-in $\tau_{\text{pre}}=0.15$ keeps 15/16, 16/16, 16/16 and still cuts wall-clock time, more as the tree widens (−5%, −6%, −17%). Computed prefill tokens fall from 1320/2624/5008 to 1018/2020/3848. Rais-

Table 7. GSM8K, Qwen3-8B, $n=16$, $D=512$, ESC only. Any-finished is exact match on a full-budget branch. $\tau_{\text{pre}}=0.15$ drops the loop; 0.45 also drops illegal text. Time is trunk warm plus fan-out, in seconds. Computed prefill falls 23% at 0.15 and 45% at 0.45 at every k .

	$k=4$	$k=8$	$k=16$
ESC	16/16, 8.65	16/16, 10.84	16/16, 17.05
APP $\tau_{\text{pre}}=0.15$	15/16, 8.22	16/16, 10.19	16/16, 14.14
APP $\tau_{\text{pre}}=0.45$	14/16, 7.89	14/16, 8.60	16/16, 10.93

Table 8. Same GSM8K slice, $k=16$, $\tau_{\text{pre}}=0.15$ on three host decoders. Admitted children and computed prefill tokens are identical across hosts: the plug-in does not read τ , d_{DPTS} , or α . Any-finished stays 16/16.

	adm.	prefill	e2e (s)	any
ESC	16	5008	17.05	16/16
ESC+APP	12	3848	14.14	16/16
Spec. Rejection	16	5008	13.14	16/16
SR+APP	12	3848	12.47	16/16
DPTS	16	5008	11.86	16/16
DPTS+APP	12	3848	11.49	16/16

ing the bar to 0.45 is the extra-speed point: −9%, −21%, −36% end-to-end, and 732/1448/2768 computed prefill tokens. It drops two questions at $k=4$ and $k=8$. At $k=16$ both bars stay at 16/16, so 0.45 is the faster matched-accuracy setting (10.93 s vs. 14.14 s). We ship 0.15 as the default because one bar must hold at every k .

Table 8 is the plug-in. ESC, Speculative Rejection, and DPTS alone all report 5008 computed prefill tokens: a decode-time cut still prefills every child. Stacking the same 0.15 bar admits 12 of 16 residuals and 3848 prefill tokens on every host. Wall-clock falls 17% on ESC, 5% on Speculative Rejection, and 3% on DPTS, because those last two hosts would have stopped the loop later anyway. The accuracy that Table 7 preserves is unchanged: every row in Table 8 is 16/16. At $k=8$ the same bar is 16/16 on ESC (10.84 to 10.19 s) and lifts Speculative Rejection and DPTS from 15/16 to 16/16.

Concurrency from measured KV. Our empirical results on DeepSeek-R1 GSM8K (761 APC tokens, 561 ForkServe tokens, $n=16$, $k=4$) in an 8 GiB pool in (10) give 76 concurrent trees for APC and 103 for ForkServe. Figure 6 sweeps offered QPS: under a 1 s P99 TTFT constraint the same model admits 384 QPS for APC and 512 for the pruned tree, and saturated decode throughput rises from 48640 to 65920 tokens/s (+35.5%). This uses measured M , not the control-plane peak of Table 4.

4.3 GPU ForkServe+

Setup. Qwen3-8B, bfloat16 , $n=80$, $D=512$, $\text{tp}=2$ and 4 on four A100-SXM4 40GB GPUs. ForkServe+ enables lazy abort, pointer swap, hash skip, draft prune, and

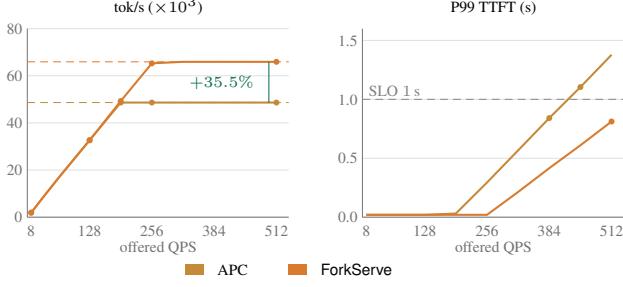


Figure 6. **SLO sweep** from measured DeepSeek GSM8K peaks. Left: saturated tok/s plateaus at $C \times \text{decode-rate}$ (76 vs. 103 slots). Right: APC crosses the 1 s P99 line at 448 QPS; ForkServe stays under it through 512.

Table 9. Qwen3-8B GPU forest, $n=80$, $D=512$. Peak KV is independent of tp. Game24 / HumanEval peaks stay with the spine.

	peak	D^+	e2e	acc.	time reduction	G24	HE
APC tp=2	1582	40960	36.0	0.900	—	1289	1970
ForkServe+ tp=2	1182	13356	28.2	0.863	0.78×	793	1882
APC tp=4	1582	40960	22.7	0.938	—	1289	1970
ForkServe+ tp=4	1182	12826	16.9	0.925	0.75×	793	1882

the P/D gate. GSM8K and Game24 are ToT ($k=4$); HumanEval is ReAct wrap+recovery ($k=2$).

Peak KV on this forest is 1182 against APC 1582 ($0.75\times$) at both widths. Game24 and HumanEval peaks are $0.62\times$ and $0.96\times$ APC.

4.4 Multi-turn Tree-of-Thoughts

Setup. Qwen3-4B and Qwen3-8B, bfloat16, tp=2, temperature 0, $n=16$. Each problem is one session of $k=4$ agents and three turns, and each system issues that problem as its own generate. Turns 1 and 2 decode a 32-token step; the losers abort; the winner spine is the parent of the next fan-out. Turn 3 is the published answer, $D=512$ on GSM8K and $D=256$ on Game24.

Depth keeps one spine. Table 10 and Figure 8 are GSM8K and Game24 under that schedule. Peak KV is 275 against APC 318 on GSM8K ($0.86\times$) and 216 against 265 on Game24 ($0.82\times$), on both models. ForkServe stays within 6% of APC end-to-end ($1.00\text{--}1.06\times$). Across 16 problems, three turns, and four agents, the expand issues 192 residuals. ForkServe+ prefills 48 of them and withholds the other 144 before they reach the GPU.

Table 11 and Figure 9 separate that expand from the winner decode. The expand speedup is $5.0\text{--}6.4\times$ for ForkServe and $7.9\text{--}10.6\times$ for ForkServe+. Decode does not match it. ForkServe decode is slower than APC because the two step decodes are timed there, which is why end-to-end stays at

Table 10. Three-turn ToT, $n=16$, $k=4$, tp=2, one problem per generate. End-to-end time is seconds per problem. Peak KV is tokens on the live spine. FS is ForkServe; FS+ is ForkServe+. Shortest end-to-end time and lowest peak in green.

	Qwen3-4B			Qwen3-8B		
	APC	FS	FS+	APC	FS	FS+
GSM8K acc.	0.938	0.938	0.938	1.000	0.938	0.938
GSM8K e2e (s)	1.66	1.76	1.63	2.48	2.56	1.91
GSM8K peak	318	275	275	318	275	275
Game24 acc.	0.375	0.500	0.562	0.562	0.562	0.625
Game24 e2e (s)	1.61	1.64	1.13	2.15	2.15	1.52
Game24 peak	265	216	216	265	216	216

Table 11. Expand versus decode, milliseconds per problem. Expand is the prefill of the k residuals across three turns. On APC that timer also includes the two 32-token step decodes; ForkServe and ForkServe+ charge those steps to decode. Speedup in Figure 9 is APC expand time over the other system. Shortest time in each trio in green.

	Expand (ms)			Decode (ms)		
	APC	FS	FS+	APC	FS	FS+
4B GSM8K	429	83	55	1215	1680	1572
4B Game24	435	87	53	1166	1552	1074
8B GSM8K	597	94	58	1867	2468	1848
8B Game24	598	94	56	1538	2060	1466

$1.00\text{--}1.06\times$. GSM8K accuracy matches APC on Qwen3-4B (15/16 on the same problems) and is one problem lower on Qwen3-8B (15/16 vs. 16/16). On Game24, ForkServe solves two more problems than APC on Qwen3-4B (8/16 vs. 6/16) and the same nine on Qwen3-8B; ForkServe+ solves 9/16 and 10/16.

4.5 Cross-model forest

Setup. Same four systems, $n=40$, $D=512$, tp=2 and 4, on Qwen3-4B, Qwen2.5-Math-1.5B, Qwen3-14B, Llama-3-8B-Instruct, Mistral-7B-Instruct, and DeepSeek-R1-Distill-Llama-8B.

Peak occupancy. APC retains the shared trunk and the union of the residuals, which is the copy-on-write term $M_{\text{APC}} = M_{\text{CoW}} = b(L + \sum_i \ell_i)$ from (1). The tree retains only the committed spine $M_{\text{spine}} = b(L + \ell_*)$. The ratio of these two terms cancels b and equals $(L + \ell_*) / (L + \sum_i \ell_i)$, so it depends on k and the lengths (L, ℓ_*) rather than on the weights or the tensor width. When the residuals have similar length, the ratio simplifies to $(L + \ell) / (L + k\ell)$ and decreases as L/ℓ decreases. Figure 7 shows this pattern: a long trunk on GSM8K gives $0.72\text{--}0.75$, and a shorter trunk on Game24 gives $0.57\text{--}0.61$. Figure 10 repeats the comparison at $n=80$ with recompute and ForkServe+ on separate axes. Relative to recompute, which stores the cloned occupancy M_{clone} , the spine is $0.25\times$. ForkServe+ matches that ratio. Accuracy remains at the APC level, which indicates that the lower peak is a storage effect on the same committed answer. Figure 11 shows the per-item decode-length distribution on

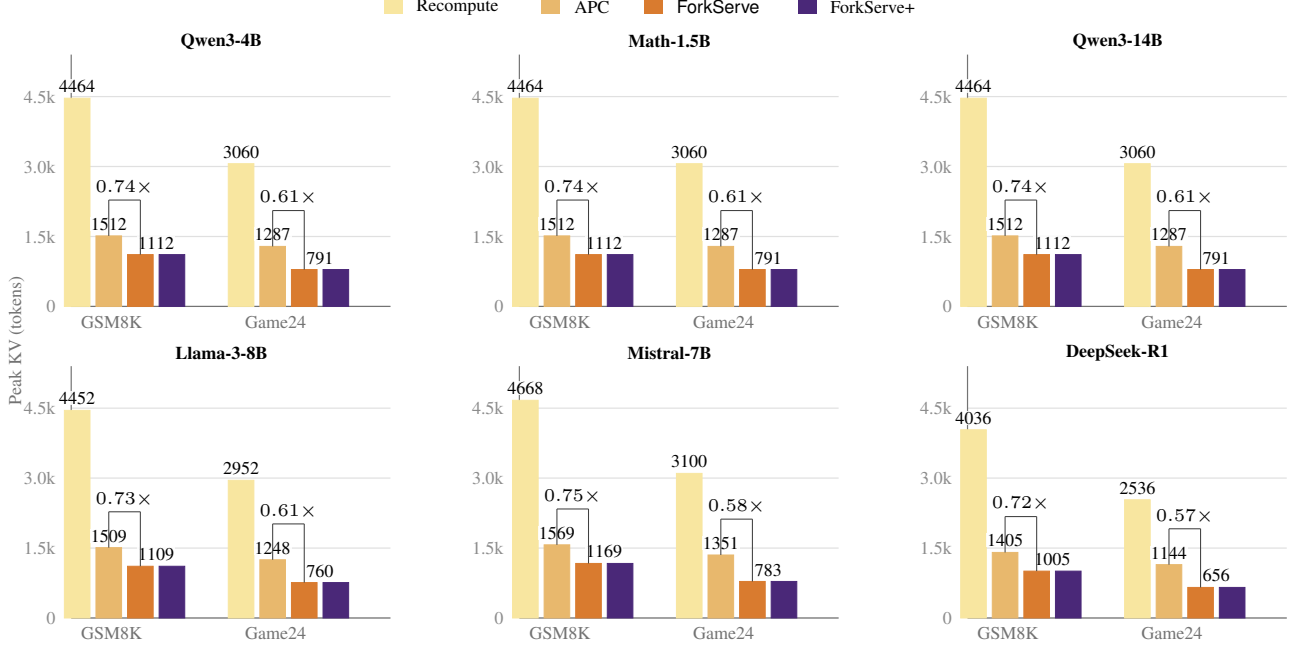


Figure 7. **Peak KV** on six models ($n=40$, $tp=2$). Each panel is one family; groups are GSM8K and Game24. Bars are recompute, APC, ForkServe, and ForkServe+ (cream / tan / orange / purple). ForkServe and ForkServe+ share the spine. Annotated $\times$ is $M_{\text{spine}}/M_{\text{APC}}$: 0.72–0.75 $\times$ on GSM8K and 0.57–0.61 $\times$ on Game24, independent of checkpoint size, as (1) predicts.

Table 12. Last-line answer stop, $n=32$, $tp=2$. ForkServe+ stops only on a finished answer in the last non-empty line. Peak is the max over chunks. GSM8K $D=512$; Game24 $D=256$. R1-8B is DeepSeek-R1-Distill-Llama-8B; L8B is Llama-3-8B; M7B is Mistral-7B. Best storage in green.

Model	Metric	APC	FS+
R1-8B	GSM8K peak	1493	1093
	GSM8K acc.	0.500	0.562
	GSM8K D^+	16384	12693
	Game24 peak	1064	616
L8B	GSM8K peak	1485	1085
	GSM8K acc.	0.719	0.688
	GSM8K D^+	16384	5101
	Game24 peak	1224	736
M7B	GSM8K peak	1521	1121
	GSM8K acc.	0.344	0.188
	GSM8K D^+	16384	6723
	Game24 peak	1296	728

GSM8K and Game24.

HumanEval uses $k=2$, so the residual is a smaller fraction of the context and M_{spine} remains close to M_{APC} . Pass@1 is unchanged across recompute, APC, and ForkServe.

4.6 Branching, pruning, and long traces

Setup. GSM8K, $n=16$, $D=512$, $tp=2$, on Llama-3-8B, Mistral-7B, and DeepSeek-R1-8B. The sweep changes k or adds a second ToT level. MATH-500 ($n=200$) and AIME

Table 13. GSM8K peak KV versus branching ($n=16$, $tp=2$, $D=512$). The spine peak is invariant in k ; the APC peak increases with k . ratio is FS/APC. L8B is Llama-3-8B; M7B is Mistral-7B; R1-8B is DeepSeek-R1-Distill-Llama-8B. Best storage in green.

Model/ratio	Method	$k=2$	$k=4$	$k=8$	depth 2
L8B	APC	610	750	998	814
	FS	550	550	550	614
FS/APC		0.90	0.73	0.55	0.75
M7B	APC	629	769	1033	841
	FS	569	569	569	641
FS/APC		0.90	0.74	0.55	0.76
R1-8B	APC	629	761	1053	841
	FS	561	561	561	641
FS/APC		0.89	0.74	0.53	0.76

($n=73$) use $D=2048$.

The ratio falls with k . Table 13 and Figure 12 measures $(L+\ell)/(L+k\ell)$ on the GPU. The spine peak is invariant in k while the APC peak increases, so the ratio moves from 0.89–0.90 at $k=2$ to 0.53–0.55 at $k=8$ on all three families. At depth 2 the ratio is 0.75–0.76. ForkServe+ leaves the peak unchanged. At $k=8$ it rejects 28 branches and reduces fan-out from 307 to 179 ms on Llama-3-8B, from 349 to 191 ms on Mistral-7B, and from 363 to 194 ms on DeepSeek-R1. With draft pruning disabled, Llama-3-8B fan-out is 247 ms and DeepSeek-R1 fan-out is 271 ms. A batch of 16 keeps the ratio at 0.72–0.73 (2071/2871 on Llama-3-8B, 2135/2935 on Mistral-7B, 2093/2893 on DeepSeek-R1).

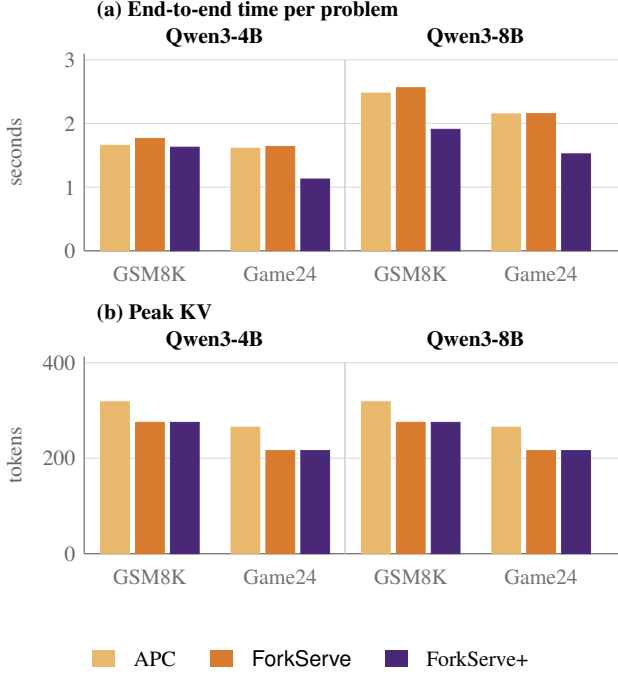


Figure 8. **Three-turn ToT** on Qwen3-4B and Qwen3-8B ($n=16$, $tp=2$, one problem per generate). (a) End-to-end seconds per problem; a shorter bar is faster. (b) Peak KV tokens. ForkServe stays with APC on latency and stores the spine ($0.86\times$ APC on GSM8K, $0.82\times$ on Game24). ForkServe+ is faster because it prefills one of four residuals.

At $D=2048$ the ratio is $0.75\text{--}0.91$ (Table 14). On MATH-500, ForkServe matches APC (432 s vs. 429 s on DeepSeek-R1). ForkServe+ cuts DeepSeek-R1 from 429 s to 247 s and Qwen3-4B from 329 s to 236 s; the Qwen3-4B $n=32$ slice is 38 s versus 53 s. ForkServe+ cuts MATH-500 fan-out from 2406 to 1284 ms on DeepSeek-R1 and from 2787 to 1014 ms on Qwen3-4B, against APC at 1593 and 1201 ms. Qwen3-4B ForkServe+ scores 120/200 on MATH-500 (APC 124/200).

5 RELATED WORK

Request-level serving. Orca [47] introduced iteration-level scheduling; vLLM [18] paged the KV cache and hashes full blocks for APC; SGLang [53] added radix prefix sharing; Sarathi [2, 3] chunked prefills; DistServe and Splitwise [54, 31] disaggregated prefill from decode. Prefix caching discovers sharing after tokens exist. Prefill-decode disaggregation isolates TTFT from inter-token latency; it does not select which branch to compute, and it transfers every page the prefiler produced. APP uses both mechanisms as filters: the hash as a skip, the connector as a gate.

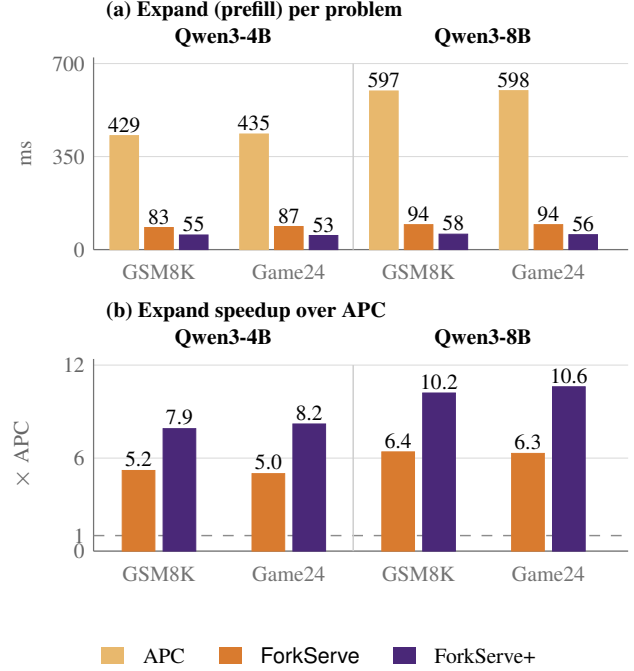


Figure 9. **Expand versus APC** on the same four runs as Table 11. (a) Expand milliseconds per problem. (b) Speedup of that expand over APC. The dashed line is $1\times$. ForkServe is $5.0\text{--}6.4\times$ and ForkServe+ is $7.9\text{--}10.6\times$. Decode in Table 11 does not repeat that ratio: the step decodes sit in the ForkServe decode column, so ForkServe end-to-end stays with APC.

Program- and SLO-aware scheduling. Autellix [25] schedules by program-level attained service; JITServe [52] allocates just-enough bandwidth under heterogeneous SLOs. Hermes [24], ThunderAgent [17], and SAGA [14] raise the atom to the program or workflow. These systems leave KV unforked and unissued children unprefilled.

KV retention and overlap. InferCept [1], Continuum [20], MORI [43], and ConServe [11] keep the *current* context alive or placed. Sutradhara [6] overlaps tool execution with prefill of one linear tool-independent prefix. SmoothAgent [30] lookahead-transforms context; IdleSpec [9] drafts plans for accuracy, not KV. Classical speculative decoding [19, 7, 22] drafts *output* tokens. ForkKV [38] CoW-splits LoRA residuals. We represent the future as a *set* of CoW children, speculate several known suffixes, commit by LCP, prune losers before prefill, and gate P/D insert.

Decoding-time pruning. A second line drops a candidate after the model has generated it. Beam search in Tree-of-Thoughts [45] scores a finished step and keeps the top beams. Early-stopping self-consistency (ESC) [21] draws a window of complete samples and skips later samples only

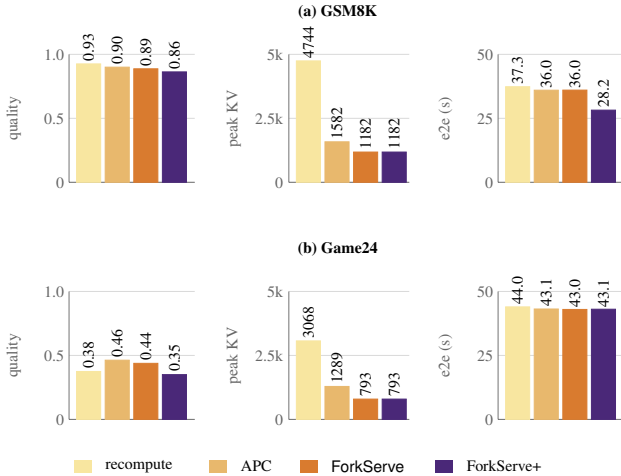


Figure 10. Four systems on Qwen3-8B ($n=80$, $tp=2$, $D=512$). Each panel has its own axis. Quality is higher-better; peak KV and end-to-end time are the measured values, so a shorter bar is better. (a) GSM8K. (b) Game24. Figure 10 shows that ForkServe and ForkServe+ lower peak KV below APC on both workloads ($0.75\times$ on GSM8K, $0.62\times$ on Game24) while GSM8K quality stays with APC, and that ForkServe+ also lowers GSM8K end-to-end time from 36.0 s to 28.2 s.

once that window agrees. Speculative Rejection [35] ranks partial generations and halts the lower quantile. Dynamic Parallel Tree Search (DPTS) [12] generates a fixed mini-step on every active node, then early-stops low-confidence children. In each method the score is computed from tokens already resident in the KV cache, so the prefill of the candidate and a decode prefix are issued before the drop. APP scores a known residual first and withholds that prefill, which also keeps the child off the disaggregated connector (§4.2). The GPU plug-in uses a frozen prefill bar and does not return when the host is ESC, Speculative Rejection, or DPTS (§4.2).

6 CONCLUSION

The KV retained for aborted branches limits the throughput and memory cost of agentic LLM serving. A Tree-of-Thoughts fan-out shares one trunk, yet a prefix cache retains the residuals of that expand, and a disaggregated connector transfers those pages when the harness later aborts them. We introduced ForkServe, which makes the branch a serving object: a copy-on-write context tree, a cascade that rejects a residual at the minimum-cost stage at which rejection is admissible, and a decode path that attends only the committed spine.

Our evaluation demonstrates a $0.72\times$ median peak KV on Qwen3-14B GSM8K at matched accuracy (0.845 vs.

Table 14. Contest forest, $D=2048$, $k=4$, $tp=2$. MATH-500 is $n=200$; AIME is $n=73$. R1 is DeepSeek-R1; 4B is Qwen3-4B. Best storage and shortest end-to-end time in green.

Model	Method	peak	/ APC	acc.	e2e (s)
R1 MATH	APC	1863	—	0.560	429
	FS	1391	0.75	0.510	432
	FS+	1599	0.86	0.545	247
R1 AIME	APC	2118	—	0.055	178
	FS	1646	0.78	0.123	178
	FS+	1662	0.78	0.096	165
4B MATH	APC	1937	—	0.620	329
	FS	1770	0.91	0.625	248
	FS+	1770	0.91	0.600	236
4B AIME	APC	2150	—	0.027	136
	FS	1794	0.83	0.041	136
	FS+	1794	0.83	0.041	131

0.844), a 74.72% cut in modeled fan-out prefill, and a 35.53% increase in the concurrency a fixed pool can sustain. Across six families the GSM8K spine stays at $0.72\text{--}0.75\times$ that peak, and the ratio falls from $0.89\text{--}0.90\times$ at $k=2$ to $0.53\text{--}0.55\times$ at $k=8$. At a 2048-token budget the ratio remains $0.75\text{--}0.91\times$, and ForkServe+ MATH-500 fan-out is 1284 ms against the prefix cache at 1593 ms on DeepSeek-R1 and 1014 ms against 1201 ms on Qwen3-4B. On Qwen3-8B, the same contest slice at $n=32$ and $D=1024$ shows why a blanket draft skip is the wrong rejection: dropping every residual below τ cuts any-finished accuracy from 20/32 to 13/32 at $k=16$, while a 128-token probe that continues only if the generated prefix scores at least τ restores 20/32 and lowers end-to-end time from 104.1 s to 72.2 s. On GSM8K ($n=16$, $D=512$) the frozen prefill plug-in $\tau_{\text{pre}}=0.15$ is the accuracy-preserving point: 15/16, 16/16, 16/16 at $k=4, 8, 16$ and -5% , -6% , -17% end-to-end against native ESC, with a 23% cut in computed prefill on ESC, Speculative Rejection, and DPTS alike. The 0.45 bar is the extra-speed point (-36% at $k=16$, still 16/16) and is the wrong default at small fan-out, where it falls to 14/16.

ForkServe’s key innovations include: (i) a copy-on-write context tree that turns `fork` into an alias of the parent’s KV pages and frees only the residual on abort, so live occupancy is the spine $b(L+\ell_*)$; (ii) advanced prefill pruning as a fixed plug-in ($\tau_{\text{pre}}=0.15$) that draft-skips repeated loops before any host decoder runs, then a probe that extends any other low-score residual only when the generated prefix clears the decode bar, and a transfer gate that inserts only the miss tail of a residual that is kept; (iii) longest-common-prefix commit, which keeps speculative tokens out of the sampler so the committed sequence matches a prefill of committed tokens only; and (iv) a two-class scheduler that admits speculative prefill only from the slack left by committed requests.

Future directions include probe lengths that track fan-out width, closed-loop QPS sweeps of the analytical concurrency gain, full-set Game24 and HumanEval forests, deeper

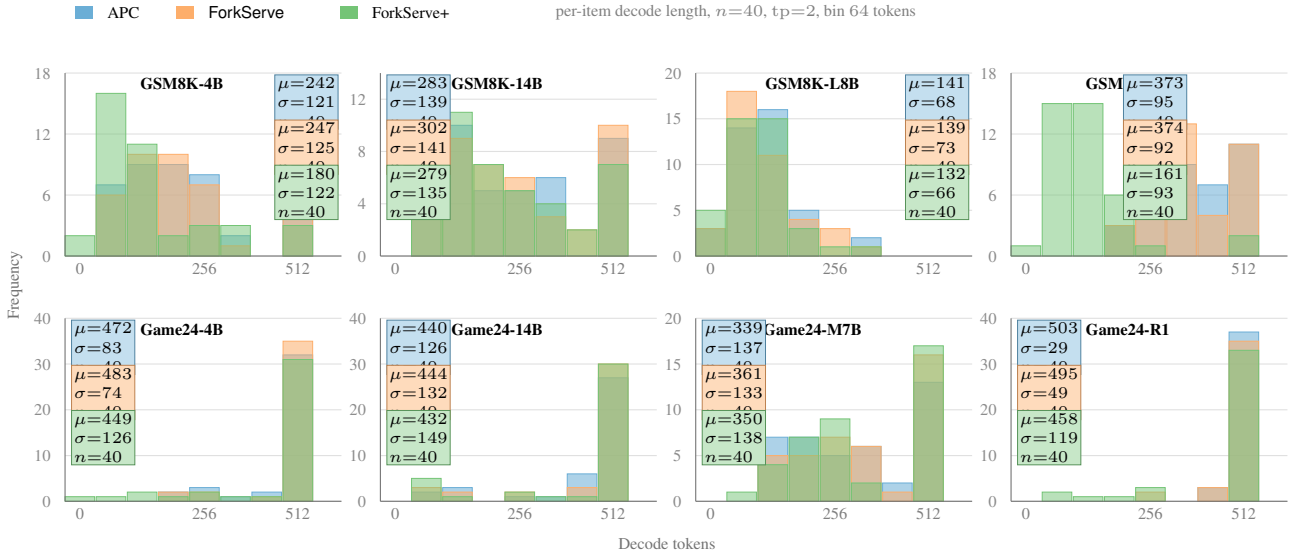


Figure 11. Per-item decode-length histograms ($n=40$, $tp=2$, $D=512$). Each panel overlays APC, ForkServe, and ForkServe+ (bin width 64). Boxes report μ , σ , and n . GSM8K (top) and Game24 (bottom) overlay APC, ForkServe, and ForkServe+. Llama-3-8B Game24 is omitted (all lengths 1); Mistral-7B is shown instead.

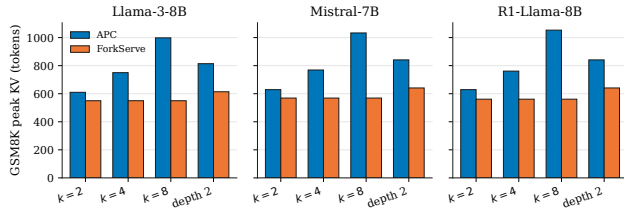


Figure 12. GSM8K peak KV versus branching ($n=16$, $tp=2$, $D=512$). ForkServe+ matches the ForkServe bar. The spine peak is invariant in k ; the APC peak grows with the hashed union, so the ratio falls from 0.89–0.90 to 0.53–0.55.

trees and 32K-context windows, and multi-tenant admission with explicit latency margins on a shared frame pool. ForkServe shows that branch cost is incurred only for the residual that is committed, and only at the minimum-cost stage in the rejection cascade: the context tree and prefill admission together yield a spine whose occupancy is set by k and residual length, independent of tensor-parallel width. As agentic workloads scale, fan-out width, tree depth, and context length all grow against a fixed KV pool. Serving systems that fork by reference and prune before prefill retain capacity on the committed path because each new branch aliases the trunk, a residual is rejected at the first admissible stage, and abort returns only that residual, so the tokens that occupy HBM are the spine the decoder attends.

REFERENCES

- [1] Reyna Abhyankar, Zijian He, Vikranth Srivatsa, Hao Zhang, and Yiyang Zhang. InferCept: Efficient Intercept Support for Augmented Large Language Model Inference. In *ICML*, 2024.
- [2] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *OSDI*, 2024.
- [3] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. SARATHI: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills. In *arXiv:2308.16369*, 2023.
- [4] Anthropic. Claude Code. <https://docs.anthropic.com/en/docs/claude-code>, 2025.
- [5] Huajun Bai, Weiwei Lv, Huichuan Zheng, Youyou Lu, and Jiwu Shu. SPORK: Self-Speculative Forking to Accelerate Agentic LLM Inference. *arXiv:2607.03333*, 2026.
- [6] Anish Biswas, Kanishk Goel, Srivarshinee S, Jayashree Mohan, Alind Khare, Anjaly Parayil, Ramachandran Ramjee, and Chetan Bansal. Sutradhara: An Intelligent Orchestrator-Engine Co-design for Tool-based Agentic Inference. *arXiv:2601.12967*, 2026.
- [7] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads. In *ICML*, 2024.
- [8] Gohar Irfan Chaudhry, Esha Choukse, Haoran Qiu, Íñigo Goiri, Rodrigo Fonseca, Adam Belay, and Ricardo Bianchini.

-
- Murakkab: Resource-Efficient Agentic Workflow Orchestration in Cloud Platforms. In *OSDI*, 2026.
- [9] Daewon Choi, Kyunghyun Park, Woomin Song, Saket Dingliwal, Sai Muralidhar Jayanthi, Jinwoo Shin, and Aram Galstyan. IdleSpec: Exploiting Idle Time via Speculative Planning for LLM Agents. *arXiv:2605.22154*, 2026.
- [10] Chroma Research. Context Rot: How Increasing Input Tokens Impacts LLM Performance. <https://research.trychroma.com/context-rot>, 2025.
- [11] Jianru Ding, Ryien Hosseini, Pouya Mahdi Gholami, Mingyuan Xiang, and Henry Hoffmann. Observation, Not Prediction: Conversation-Level Disaggregated Scheduling for Agentic Serving. *arXiv:2606.01839*, 2026.
- [12] Yifu Ding, Wentao Jiang, Shunyu Liu, Yongcheng Jing, Jinyang Guo, Yingjie Wang, Jing Zhang, Zengmao Wang, Ziwei Liu, Bo Du, Xianglong Liu, and Dacheng Tao. Dynamic Parallel Tree Search for Efficient LLM Reasoning. In *ACL*, 2025.
- [13] In Gim, Zhiyao Ma, Seung-seob Lee, and Lin Zhong. Pie: A Programmable Serving System for Emerging LLM Applications. In *SOSP*, 2025.
- [14] Dongxin Guo, Jikun Wu, and Siu-Ming Yiu. SAGA: Workflow-Atomic Scheduling for AI Agent Inference on GPU Clusters. In *HPDC*, 2026.
- [15] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. *ICLR*, 2024.
- [16] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *ICLR*, 2024.
- [17] Hao Kang, Ziyang Li, Xinyu Yang, Weili Xu, Yinfang Chen, Junxiong Wang, Beidi Chen, Tushar Krishna, Chenfeng Xu, and Simran Arora. ThunderAgent: A Simple, Fast and Program-Aware Agentic Inference System. *arXiv:2602.13692*, 2026.
- [18] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *SOSP*, 2023.
- [19] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast Inference from Transformers via Speculative Decoding. *ICML*, 2023.
- [20] Hanchen Li, Runyuan He, Qiuyang Mang, Qizheng Zhang, Huanzhi Mao, Xiaokun Chen, Hangrui Zhou, Alvin Cheung, Joseph Gonzalez, and Ion Stoica. Continuum: Efficient and Robust Multi-Turn LLM Agent Scheduling with KV Cache Time-to-Live. *arXiv:2511.02230*, 2025.
- [21] Yiwei Li, Peiwen Yuan, Shaoxiong Feng, Boyuan Pan, Xinglin Wang, Bin Sun, Heda Wang, and Kan Li. Escape Sky-High Cost: Early-Stopping Self-Consistency for Multi-Step Reasoning. In *ICLR*, 2024.
- [22] Yuhong Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty. In *ICML*, 2024.
- [23] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Huang. Parrot: Efficient Serving of LLM-based Applications with Semantic Variable. In *OSDI*, 2024.
- [24] Yifei Liu, Zuo Gan, Zhenghao Gan, Weiye Wang, Chen Chen, Yizhou Shan, Xusheng Chen, Zhenhua Han, Yifei Zhu, Shixuan Sun, and Minyi Guo. Efficient Serving of LLM Applications with Probabilistic Demand Modeling. *arXiv:2506.14851*, 2025.
- [25] Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E. Gonzalez, and Ion Stoica. Autellix: An Efficient Serving Engine for LLM Agents as General Programs. *arXiv:2502.13965*, 2025.
- [26] Zizhang Luo, Yuhao Luo, Youwei Xiao, Yansong Xu, Runlin Guo, and Yun Liang. Hive: A Multi-Agent Infrastructure for Algorithm- and Task-Level Scaling. *arXiv:2604.17353*, 2026.
- [27] Grégoire Mialon, Clémentine Fourier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: a Benchmark for General AI Assistants. *arXiv:2311.12983*, 2023.
- [28] Hongqiu Ni, Jiabao Zhang, Guopeng Li, Zilong Wang, Ruiqi Wu, Chi Zhang, and Haisheng Tan. Astraea: A State-Aware Scheduling Engine for LLM-Powered Agents. *arXiv:2512.14142*, 2025.
- [29] Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. MemGPT: Towards LLMs as Operating Systems. *arXiv:2310.08560*, 2023.
- [30] Zaifeng Pan, Qianxu Wang, Zhengding Hu, Chang Chen, Yue Guan, Yanbo Zhou, Steven Swanson, and Yufei Ding. SmoothAgent: Efficient Long-Horizon LLM-Based Agent Serving with Lookahead Context Engineering. *arXiv:2607.00151*, 2026.
- [31] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *ISCA*, 2024.
- [32] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large Language Model Connected with Massive APIs. *NeurIPS*, 2024.
- [33] Dennis M. Ritchie and Ken Thompson. The UNIX Time-Sharing System. *CACM*, 17(7), 1974.
- [34] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language Models Can Teach Themselves to Use Tools. *NeurIPS*, 2023.
- [35] Hanshi Sun, Momin Haider, Ruiqi Zhang, Huitao Yang, Jiahao Qiu, Ming Yin, Mengdi Wang, Peter Bartlett, and Andrea Zanette. Fast Best-of-N Decoding via Speculative Rejection. In *NeurIPS*, 2024.

-
- [36] Xin Tan, Yimin Jiang, Yitao Yang, and Hong Xu. Towards End-to-End Optimization of LLM-based Applications with Ayo. In *ASPLOS*, 2025.
- [37] Andrew S. Tanenbaum and Herbert Bos. *Modern Operating Systems*. Pearson, 4 edition, 2015.
- [38] Shao Wang, Rui Ren, and Lin Gui. ForkKV: Scaling Multi-LoRA Agent Serving via Copy-on-Write Disaggregated KV Cache. *arXiv:2604.06370*, 2026.
- [39] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xian-gru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muen-nighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. Open-Hands: An Open Platform for AI Software Developers as Generalist Agents. *ICLR*, 2025.
- [40] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-Consistency Improves Chain of Thought Reasoning in Language Models. *ICLR*, 2023.
- [41] Ying Wang, Zhen Jin, Zhenqian Chen, Jiexiong Xu, Wenhai Lin, Yiquan Chen, and Wenzhi Chen. AugServe: Adaptive Request Scheduling for Augmented Large Language Model Inference Serving. In *ICML*, 2026.
- [42] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv:2308.08155*, 2023.
- [43] Tian Xia, Hanchen Li, Zhifei Li, Xiaokun Chen, Hao Kang, Yifan Qiao, Yi Xu, and Ion Stoica. Idleness is Relative: Exploiting Tool-Call Idle Windows for Offloading in Agentic Systems with MORI. *arXiv:2606.00866*, 2026.
- [44] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *NeurIPS*, 2024.
- [45] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. *NeurIPS*, 2023.
- [46] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. *ICLR*, 2023.
- [47] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *OSDI*, 2022.
- [48] Shan Yu, Junyi Shu, Yuanjiang Ni, Kun Qian, Xue Li, Yang Wang, Jinyuan Zhang, Ziyi Xu, Shuo Yang, Lingjun Zhu, Ennan Zhai, Qingda Lu, Jiarong Xing, Youyou Lu, Xin Jin, Xuanzhe Liu, and Harry Xu. Pythia: Exploiting Workflow Predictability for Efficient Agent-Native LLM Serving. *arXiv:2604.25899*, 2026.
- [49] Yichao Yuan, Mosharaf Chowdhury, and Nishil Talati. KAIROS: Stateful, Context-Aware Power-Efficient Agentic Inference Serving. *arXiv:2604.16682*, 2026.
- [50] Matei Zaharia, Omar Khattab, Lingjiao Chen, Jared McCauley, et al. The Shift from Models to Compound AI Systems. *Berkeley AI Research Blog*, 2024.
- [51] Di Zhang et al. Accessing GPT-4 Level Mathematical Olympiad Solutions via Monte Carlo Tree Self-refine. *arXiv:2312.11324*, 2023.
- [52] Wei Zhang, Zhiyu Wu, Yi Mu, Rui Ning, Banruo Liu, Nikhil Sarda, Myungjin Lee, and Fan Lai. JITServe: SLO-aware LLM Serving with Imprecise Request Information. In *NSDI*, 2026.
- [53] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient Execution of Structured Language Model Programs. In *NeurIPS*, 2024.
- [54] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *OSDI*, 2024.
- [55] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abhijit Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A Realistic Web Environment for Building Autonomous Agents. *ICLR*, 2024.